

README

Part 10: 分布式训练 — 从单卡到多卡

- > Part 8 训练的后训练模型、Part 9 打开的 GPU 引擎盖——都建立在一个前提上：单卡装得下。
- > 真实的 LLM 训练动辄几百上千张卡。这一部分回答：多卡为什么不是"买个交换机插上就行"，
- > DDP/ZeRO/FSDP/张量并行/流水线并行各自解决什么问题、代价是什么。
- > 参考来源：[pytorch/examples minGPT-ddp](https://github.com/pytorch/examples/tree/main/distributed/minGPT-ddp)、
- > PyTorch DDP/FSDP 官方系列、[huggingface/nanotron](https://github.com/huggingface/nanotron) 与
- > [Ultra-Scale Playbook](https://huggingface.co/spaces/nanotron/ultrascale-playbook)。

学习目标

完成本部分后，你将能够：

- 预测 broadcast / all_reduce / all_gather / reduce_scatter 四个集合通信原语的输出，说清 DDP 梯度同步"平均不是求和"的语义（01 章）
- 写出 DDP 五件套与 **no_sync** 梯度累积，推导"多卡一步 == 大 batch 一步"，画出桶化 all-reduce 与 backward 重叠的时序（02 章）
- 背出并推导 16Ψ 显存账本与 ZeRO 1/2/3 每卡公式 ($4\Psi+12\Psi/N \rightarrow 2\Psi+14\Psi/N \rightarrow 16\Psi/N$)，用决策树为任意模型/卡数选并行策略（03 章）
- 画出 Megatron TP 的列/行切法与 GPipe/1F1B 时间线，推导 $\text{bubble}=(p-1)/(m+p-1)$ ，读懂 3D 并行的工业配置（04 章，进阶可选）

章节导航

章节	内容
为什么并行 + 分布式 Hello World](01_why_and_collectives.md)	并行分类、rank/world_size 心智模型、集合通信原语、torchrun 报错 FAQ
DDP：数据并行深入](02_ddp.md)	梯度平均、桶化 all-reduce、DistributedSampler、no_sync、吞吐实测
显存账本与 ZeRO/FSDP](03_memory_zero_fsdp.md)	16Ψ 公式、ZeRO 1/2/3、FSDP 实战与显存对比
张量并行、流水线并行与工业栈](04_tp_pp_and_beyond.md)（进阶可选）	Megatron f/g 算子、GPipe/1F1B、bubble 公式、3D 并行、LLaMA 训练配置

前置知识

- 必须掌握：
- [Part 8 01 章：GPT-2 与预训练](../Part8_post_training/tutorial/01_gpt_and_pretrain.md)——训练循环与 AdamW。DDP/FSDP 改变的只是"梯度从哪来、状态存在哪"，单卡训练语义是全部讨论的地基
- 建议掌握：
- [Part 9 01 章：GPU 架构与第一个 CUDA 内核](../Part9_cuda_kernels/tutorial/01_gpu_and_first_kernel.md)——知道"内核异步执行"即可；02 章讲通信与 backward 重叠、MFU 时会用到

- [Part 7 03 章：GQA 与 FFN](../Part7_minimind/tutorial/03_gqa_and_ffn.md)——04 章 TP"按注意力头切分"直接引用这里的多头结构

可选：

- [Part 6 03 章：Transformer Block](../Part6_transformer/tutorial/03_transformer_block.md)——想对照"一个 Block 如何成为 FSDP/PP 的分片单元"时参考
- 不需要任何分布式经验——01 章从零建立 SPMD 心智模型

学习路线图

Part 8/9（单卡：模型装得下、算得动）
|
| "模型 70B 装不下 / 数据大到单卡喂不饱 / 想要线性加速"



Part 10: 分布式训练
① 心智模型 + 集合通信 — rank/collective/torchrun ——→ 01_why_and_collectives.md
② DDP — 梯度平均 + 通信重叠（数据并行） ——→ 02_ddp.md
③ 显存账本 + ZeRO/FSDP — 把"模型状态"切开来存 ——→ 03_memory_zero_fsdp.md
④ TP / PP（进阶） — 把"一层计算"切开算 / 按层流水 ——→ 04_tp_pp_and_beyond.md

环境要求与"无多卡学习路径"

全部脚本单进程可直接跑（自动退化为 world_size=1 的 gloo，CPU 也行）；
多卡验证用 torchrun --standalone --nproc_per_node=2 脚本名。

	能做什么
	脚本 01/02/03/05/06 全部可跑（gloo，多进程也支持：`torchrun --nproc_per_node=2` 不要求 GPU！）
	以上全部 + 04 的 FSDP（单卡分片，显存对比意义有限但能跑通）
U	完整体验：02 的吞吐对比、04 的跨卡分片、05/06 的切分验证
想体验多卡	**CPU 多进程同样演示分布式语义**：`torchrun --standalone --nproc_per_node=2 01_xxx.py`（gloo 后端）——并行逻辑与 GPU 完全一致，只

本课开发机验证环境：RTX 4090×2 + torch 2.6.0+cu124 + NCCL。所有"验收数字"（吞吐对比、显存对比、loss 一致性）都来自该环境实测。

一张表看懂四种并行

并行	切什么	通信	解决	代价
数据并行 DDP	数据	每步 all-reduce 梯度	数据太大 / 加速	每 rank 存完整模型

并行	切什么	通信	解决	代价
ZeRO/FSDP	模型状态（参数/梯度/优化器）	all-gather + reduce-scatter	模型状态装不下	通信 $\approx 1.5\times$
张量并行 TP	单层权重矩阵	每层 2 次 all-reduce	单层放不下 / 大激活	通信最频繁，只适合机内
流水线并行 PP	按层分组	点对点传激活	层数太多装不下	bubble 空转 $(p-1)/(m+p-1)$

课后作业

每章末尾有思考题。全部学完后：

[Assignment 10](../assignments/assignment_10/)（题 1-4 必做、纯 CPU 可完成：
all-reduce 语义 / 显存账本计算器 / DistributedSampler 证明 / TP 分块数学；
题 5 可选：流水线 bubble——未实现时测试自动 SKIP，不算失败）

相关资源

- [pytorch/examples minGPT-ddp](https://github.com/pytorch/examples/tree/main/distributed/minGPT-ddp) — Karpathy 原 minGPT-ddp 的官方维护版
- [PyTorch DDP 设计笔记](https://docs.pytorch.org/docs/stable/notes/ddp.html) · [FSDP 教程](https://docs.pytorch.org/tutorials/intermediate/FSDP_tutorial.html)
- [huggingface/nanotron](https://github.com/huggingface/nanotron) — 3D 并行极简实现
- [Ultra-Scale Playbook](https://huggingface.co/spaces/nanotron/ultrascale-playbook) — HF 的集群训练系统教程（本课的"下一站"）
- [ZeRO (arXiv 1910.02054)](https://arxiv.org/abs/1910.02054) · [Megatron-LM (1909.08053)](https://arxiv.org/abs/1909.08053) · [Reduced Activation Re-computation (2205.05198)](https://arxiv.org/abs/2205.05198)

[← 上一章：Part 9 CUDA 内核](../Part9_cuda_kernels/tutorial/README.md) | [下一章：Part 11 verl 对齐实战 →](../Part11_alignment_verl/tutorial/README.md)

01_why_and_collectives

01 — 为什么并行 + 分布式 Hello World

- > 单卡到多卡，最难跨过的不是 API，而是心智模型：同一份代码被 N 个进程同时执行。
- > 本章先建立这个模型，再用四个集合通信原语（脚本 01 亲手验证）补上分布式训练的"字母表"，
- > 最后给一份 torchrun 常见报错 FAQ——01 章不卡人，是硬性设计目标。

学习目标

完成本章后，你将能够：

- 解释"三个装不下/不够"分别对应哪种并行，以及学习顺序为什么是 DDP → ZeRO → TP/PP
- 建立 SPMD 心智模型：一份代码被 N 个进程执行，rank/world_size 区分身份
- 预测 broadcast / all_reduce / all_gather / reduce_scatter 四个原语的输出（并用脚本 01 验证）

- 说出 DDP 梯度同步的语义是平均不是求和（一个数记住整章）
- 排查 torchrun 六类常见报错（症状 → 原因 → 解法）

前置知识

必须掌握：

- Part 8 的训练循环（知道 optimizer.step() 干什么即可）

建议掌握：

- 多进程与环境变量的基本概念（RANK / WORLD_SIZE 本章边用边讲，不需要分布式经验）

1. 三个"装不下/不够"，三种并行

痛点	症状	药方
数据喂不饱 / 想要加速	GPU 利用率高但训完要一个月	**数据并行 DDP** （每卡不同数据，梯度同步）
模型状态装不下	7B 模型 AdamW 要 16B/参数 → $7B \times 16 = 112GB$	**ZeRO / FSDP** （把模型状态切片）
单层激活/权重放不下	巨大 embedding、超宽 FFN	**张量并行 TP** （把一层矩阵切开算）
层数太多装不下	$406M \times 24$ 层 + 激活	**流水线并行 PP** （按层分组接力）

真实 LLM 训练是它们的组合（"3D 并行"）： $TP \times PP \times DP$ ，LLaMA 2 70B = 8 路 $TP \times \dots$ （04 章细讲）。学习顺序按"通信代价从小到大"：DDP → ZeRO/FSDP → TP/PP。

2. 心智模型：SPMD

分布式训练的主流形态是 SPMD（Single Program, Multiple Data）：

你只写一份程序，torchrun 把它复制成 N 份进程同时启动：

进程 0（rank=0）：跑 main()

进程 1（rank=1）：跑同一个 main()

...

唯一区别：每个进程从环境变量里读到的 rank 不同 → 用 rank 决定"我处理哪份数据/哪片权重"

进程之间不传"消息"，而是做【集合通信】（collective）——N 个进程对齐的集体操作

- `world_size` = 总进程数；`rank` = 进程编号（0 起）；`local_rank` = 本机内的编号（多机时用）。
 - 初始化：`dist.init_process_group(backend)`。backend 选择：GPU 用 NCCL（快），CPU 用 gloo（兼容性好）。torchrun 注入 `RANK/WORLD_SIZE/MASTER_ADDR/MASTER_PORT`，`env://` 方式自动读取——这就是为什么脚本第一行经常是 `os.environ["RANK"]`。
 - 脚本 01 的 `setup()` 展示了"单进程也能跑"的技巧：没有 RANK 时自己伪装 `world_size=1`。
- 本课全部脚本兼容两种启动，这也是给你自己的分布式代码调试建议：先用单进程调通逻辑，再加卡。

3. 集合通信字母表（跑 `scripts/01_distributed_basics.py`）

原语	语义	训练中的用途
`broadcast`	rank0 → 所有人	开场同步权重；DDP 构造时 broadcast state_dict
`all_reduce`	所有 rank 各持一份 → 规约 (SUM/AVG...) → 人人有结果	**DDP 梯度同步** (默认等价于取平均)
`all_gather`	人人有数据 → 拼出所有人的	FSDP forward 前收齐参数分片
`reduce_scatter`	规约后按 rank 切片分发	FSDP backward 分片梯度

- 最容易错的一点：DDP 的 all-reduce 语义是平均不是求和（脚本里 SUM 后 ÷ N 演示）。所以"多卡有效 batch = 每 rank batch × N"时，学习率的参照系是有效 batch，不是单卡 batch。
- ⚠ 集合通信是集体对齐的：任何一个 rank 少调一次 all_reduce，其他 N-1 个进程会永远等它（表现为程序卡住不报错）。调试卡死第一反应：各 rank 的执行路径是否分叉了。

脚本 01 的实测（双卡）：

```
[2] all_reduce(SUM): rank i 持 i+1 → 人人得到 [3,3,3,3]
all_reduce(SUM)/world = 梯度平均 [1.5, ...]
[4] reduce_scatter(SUM): 规约后按 rank 切片，rank0 得 [1, 3]
```

4. torchrun 报错 FAQ（01 章必读，省你一晚上）

症状	原因	解法
`Connection reset by peer` / 卡在 init	MASTER_PORT 被占（上个进程没退干净）	`--master_port=29501` 换端口；`pkill -f torchrun`
`NCCL error` / invalid usage	驱动/拓扑/容器 IPC 问题	先换 `backend='gloo'` 定位是不是 NCCL 的锅
程序卡住不报错	某 rank 没进同一个 collective	检查 if rank==0 分支里是否漏了别人的集体操作
`Cannot use ReduceOp.AVG with Gloo`	gloo 不支持 AVG	用 `SUM` 再 `/world`（本课脚本 02 踩过）
两台机器连不上	MASTER_ADDR 写了 localhost	rank0 的真实 IP；防火墙放行 MASTER_PORT
每个 epoch 数据完全相同	忘了 `sampler.set_epoch(epoch)`	DistributedSampler 用 epoch 做打乱种子

- > 本课机器实测还有一个彩蛋坑：这台 4090+4090D 的混合机型上 **NCCL 的 send/recv 点对点会互相卡死**（脚本 06 真踩到），而集合通信正常。脚本 06 的解法是给点对点单独建 gloo 组、
- > 张量过 CPU 中转——遇到类似"集合通信正常但 p2p 卡死"，这是可抄的工程解法。

学完本章你能...

- 说出四种并行各自解决的"装不下/不够"，以及学习顺序为什么是 DDP→ZeRO→TP/PP
- 解释 SPMD：一份代码、N 个进程、rank 区分身份、集合通信对齐
- 手写四个集合通信原语的语义（并用脚本 01 验证过）
- 背出 torchrun 六大常见报错的"症状→解法"

课后练习

```
<details>
<summary>Q1: world_size=4, 每 rank local batch=8, DDP 一步等效于单卡多大的
batch? 梯度是什么的梯度? </summary>
A: 等效 32。all-reduce 平均后，每个参数的梯度 = 4 个 rank 各自 batch=8 梯度的平均 = 完整
batch 32 的平均梯度。所以 DDP 不是"4 个独立训练"也不是"求和"，而是"一个大 batch 被切开展"。
</details>
```

<details>

<summary>Q2: 为什么集合通信"少调一次"是卡死而不是报错？</summary>

A: all_reduce 等操作是 N 个进程的握手协议：每个 rank 到齐才开始交换。少一个人，其余人的等待永不超时（默认无超时或超长）。这也是为什么"分 rank 的 if 分支"里要格外小心——分支里也要保证集体操作的对齐。

</details>

<details>

<summary>Q3: gloo 和 NCCL 各是什么？什么时候必须用哪个？</summary>

A: NCCL 是 NVIDIA 的 GPU 集合通信库（走 NVLink/PCIe/RDMA，GPU 张量最快）；gloo 是 PyTorch 的通用后端（CPU/GPU 都行，兼容性最好）。GPU 集合通信首选 NCCL；CPU 或排障时用 gloo。点对点 send/recv 在个别机器拓扑上 NCCL 有坑，可建 gloo 组兜底（脚本 06 实测）。

</details>

课后作业

[Assignment 10](../assignments/assignment_10/) 题 1 (all-reduce 语义)

下一步

字母表齐了。现在把最常用的原语 all-reduce 用在真正的大事上——多卡训练一个 GPT，并搞懂 DDP 内部怎么把通信藏进 backward 的间隙里。

[02 — DDP：数据并行深入](02_ddp.md)

02_ddp

02 — DDP：数据并行深入

- > DDP (DistributedDataParallel) 是所有分布式训练的地基，也是 90% 场景的正确起点。
- > 本章走完它的五个要件，用三步推导看清"all-reduce 平均 == 大 batch 梯度"，
- > 看穿内部机制（桶化 all-reduce 与 backward 重叠），并用双卡实测吞吐。
- > 跑 [scripts/02_ddp_gpt.py](../scripts/02_ddp_gpt.py)。

学习目标

完成本章后，你将能够：

- 写出 DDP 五件套，并避开两个高频 bug (DataLoader 双重打乱、忘 `set_epoch`)
- 推导"all-reduce 平均后的梯度 == 大 batch 梯度" (§ 2)，说清等价成立的三个前提
- 画出桶化 all-reduce 与 backward 重叠的时序图，解释"多卡近线性加速"从哪来
- 组合梯度累积与 `no_sync`：累积步不通信、最后一步只同步一次
- 解读多卡吞吐数字——为什么 toy 模型的加速比不可信、MFU 该用真实规模测

前置知识

必须掌握：

- 01 章：SPMD 心智模型、all_reduce 平均语义、torchrun

建议掌握：

- [Part 9 01 章](../Part9_cuda_kernels/tutorial/01_gpu_and_first_kernel.md)："内核异步执行"—理解桶化 all-reduce 与 backward 重叠的前提

1. DDP 五件套（脚本 02 的骨架）

```
`python
```

① 进程组（torchrun 注入 RANK/WORLD_SIZE/MASTER_*, env:// 自动读取）

```
dist.init_process_group(backend="nccl") # 单机多卡；CPU 用 gloo
rank = dist.get_rank()
torch.cuda.set_device(rank) # 一卡一进程，各绑各的 GPU
```

② 数据分片：DistributedSampler（不是把 batch 切开，是 dataset 切开！）

```
sampler = DistributedSampler(dataset, num_replicas=world, rank=rank, shuffle=True)
loader = DataLoader(dataset, batch_size=16, sampler=sampler, shuffle=False) # ⚠ shuffle=False
```

③ 包装模型（rank0 的权重自动 broadcast 给所有 rank → 起点一致）

```
ddp_model = DDP(model, device_ids=[rank])
```

④ 训练循环：每轮 set_epoch（sampler 用 epoch 做打乱种子）

```
for epoch in range(E):
    sampler.set_epoch(epoch)
    for xb, yb in loader:
        _, loss = ddp_model(xb, yb) # DDP 透传 forward
        loss.backward() # ← all-reduce 在这里自动发生
    opt.step(); opt.zero_grad()
```

⑤ 收尾

```
dist.destroy_process_group()
`
```

- ⚠ 两个高频 bug：DataLoader 里忘写 `shuffle=False`（与 sampler 双重打乱）；忘写 `sampler.set_epoch`（每个 epoch 数据分片完全相同，等效于只在 1/world 的数据上训练 N 遍）。

2. 数学：为什么"all-reduce 平均后 == 大 batch 一步"

五件套注释里那句"all-reduce 在 backward 里自动发生"，藏着一个值得摊开的数学事实：DDP 走一步，严格等于"用大 batch 单卡走一步"。设 N 个 rank、每 rank 本地 batch b ，总有效 batch $B = N \cdot b$ ，全局损失是 B 个样本损失的平均：

$$L(\theta) = (1/B) \cdot \sum_{i=1..B} \ell_i(\theta) \quad \# \ell_i: \text{第 } i \text{ 个样本的交叉熵}$$

第 1 步（求导穿过求和号）：梯度对样本损失是线性的——和的导数 = 导数的和：

$$\nabla L = (1/B) \cdot \sum_{i=1..B} \nabla \ell_i$$

第 2 步（按 rank 把样本分组）：DistributedSampler 把 B 个样本不重不漏地切成 $D_0, D_1, \dots, D_{N-1}$ （每份恰好 b 个）。大求和就可以"先组内、再组间"地重排：

$$\begin{aligned} \nabla L &= (1/(N \cdot b)) \cdot \sum_{r=0..N-1} \sum_{i \in D_r} \nabla \ell_i \\ &= (1/N) \cdot \sum_{r=0..N-1} \left[(1/b) \cdot \sum_{i \in D_r} \nabla \ell_i \right] \end{aligned}$$

g_r : rank r 的"本地 batch 梯度"

第 3 步（对照 DDP 实际做的事）：rank r 的 `F.cross_entropy(...)` 默认 `reduction='mean'`，本地 loss 正是 $(1/b) \cdot \sum_{i \in D_r} \ell_i$ ，backward 得到的就是 g_r ；DDP 对每个桶做 all-reduce(SUM) 后 $\div N$ ，于是每个 rank 最终拿到：

$$(1/N) \cdot \sum_{r=0..N-1} g_r == \nabla L \quad \# \text{与大 batch 单卡一步逐参数相等}$$

等价成立依赖三个前提，缺一个都不成立：

	谁负责	破坏时的后果
不重不漏且等大**	DistributedSampler + DataLoader `drop_last`	不等大 $\rightarrow$ "平均的平均 $\neq$ 总平均"（见下方直觉）；重复/遗漏样本 $\rightarrow$ 平均的根本不是
取**均值**	`reduction='mean'`（默认）	若是求和， g_r 大 b 倍，等效学习率放大
从**同一权重**出发	DDP 构造时 broadcast	各练各的，见 Q1

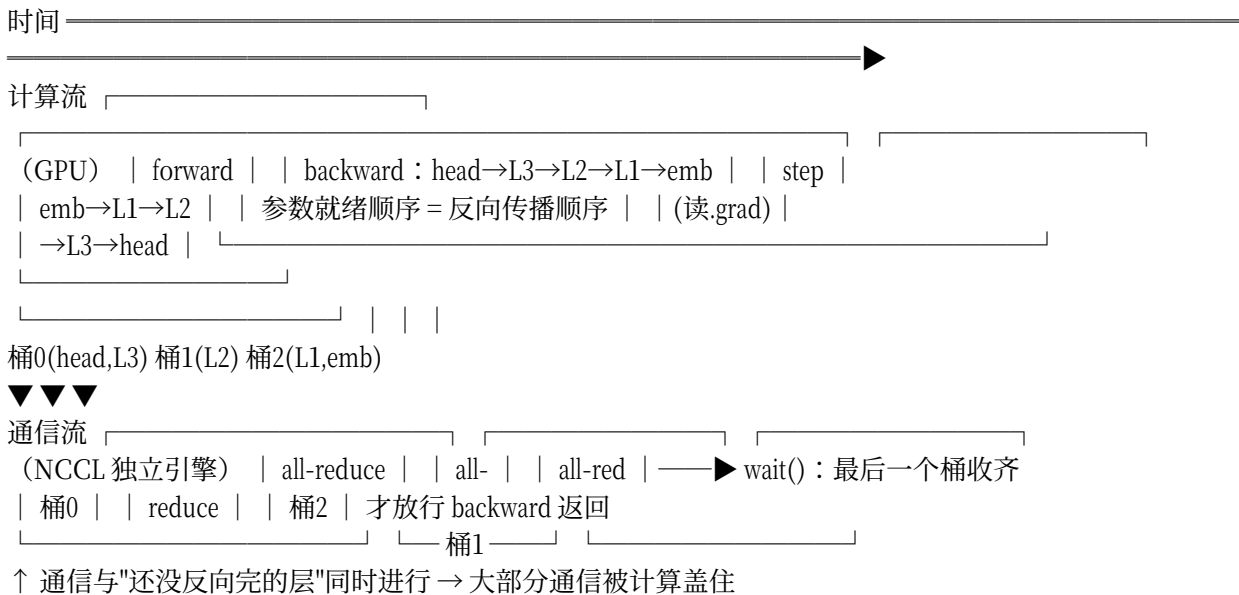
- > 直觉：梯度是线性的。整条推导只用了一件事——样本梯度可以任意分组、先组内求平均再组间求平均。所以"等大分组的平均的平均"完全等于"对全体直接取平均"。
- > 唯一能打破它的是组不等大： $(1/N) \cdot \sum g_r$ 给每个 rank 等权，而总平均按样本数加权——
- > 小组里的样本会被"超权"。这正是 DistributedSampler 宁可复制/丢弃少量样本也要把每份
- > 切得等大的数学原因（样本数不整除 world 时它复制少量样本补齐等大；`drop_last=True`
- > 丢掉凑不满一个 micro-batch 的尾巴，同理）。动手 2（本章末）让你用 10 行代码亲眼
- > 看到"等大成立、不等大失效、加权能救回来"。

3. 内部机制：通信是怎么"藏"进 backward 的

朴素实现是"forward $\rightarrow$ backward 全算完 $\rightarrow$ all-reduce 所有梯度 $\rightarrow$ step"，通信完全串行。DDP 的真实做法（[设计笔记](https://docs.pytorch.org/docs/stable/notes/ddp.html)）：

构造时：按参数的反向传播【就绪顺序】把参数分桶（默认桶 25MB）
backward 时：某个桶的全部梯度一就绪 → 立刻异步 all-reduce 这个桶
↓
通信与"剩余层的 backward 计算"重叠 → 大部分通信时间被计算盖住
全部桶发起后，backward 结束前才阻塞等最后一个桶

把一次 iteration 画成"计算/通信"两条流（world_size=2，桶按反向就绪顺序编号，示意 3 桶）：



- 这就是"多卡吞吐接近线性"的原因：通信不是没有，而是被藏起来了。
顺带记住面试常问的 MFU (Model FLOPs Utilization) = 实测 FLOPS ÷ 卡的峰值 FLOPS——
评估分布式训练效率的标准指标，第四部分会反复用到。
- 两个相关参数：`find_unused_parameters=True`（有未参与 loss 的参数时防挂死，有遍历开销）；
`broadcast_buffers=True`（默认开：每次 forward 前 rank0 的 buffer 广播给所有 rank——
BatchNorm 的 running stats 靠它同步。这也解释了 DDP+BN 的行为）。
- 对照脚本 02 的真实规模：它的 GPT 只有 628,161 个参数（fp32 ≈ 2.5MB），远小于默认
桶上限 25MB——DDP 实际只建了 1 个桶（backward 后用
`len(ddp.reducer._get_zeros_like_grad_buckets())` 可验证；实测环境 RTX 4090×2,
torch 2.6.0+cu124, NCCL）。所以"桶间重叠"在 toy 规模上根本无从体现，§ 5 里双卡吞吐
与单卡持平才是符合预期的读数；真实大模型单层参数就是 GB 级，桶化+重叠才是吞吐生命线。
- 从 no_sync 的视角再看这张图：前 K-1 个 micro-step，通信流上一格都不发；第 K 步的
backward 才把上面这组 all-reduce 一次性发出（数学见 § 4）。

4. 梯度累积：no_sync

梯度累积（小卡模拟大 batch）与 DDP 组合有个坑：默认每次 backward 都 all-reduce，
累积 4 步就通信 4 次，其中 3 次是浪费。正确姿势：

```
python
for it, (xb, yb) in enumerate(loader):
    is_last = (it % accum == accum - 1)
    ctx = ddp_model.no_sync() if not is_last else nullcontext()
```

```

with ctx:
_, loss = ddp_model(xb, yb)
(loss / accum).backward() # no_sync 内：只累积本地梯度，不通信
if is_last:
opt.step(); opt.zero_grad() # 最后一步的 backward 才 all-reduce

```

数学：K 步累积后一次 all-reduce == $K \cdot N \cdot b$ 大 batch 的梯度

目标 batch 是 $K \cdot N \cdot b$ (K 个 micro-step $\times$ N 卡 $\times$ 本地 b)。把 §2 的结论再用一次——先在第 k 个 micro-step 内部对 N 卡平均，再对 K 步平均（还是那件事：梯度是线性的）：

$$\nabla L_{\text{big}} = (1/K) \cdot \sum_{k=1..K} \hat{g}_k, \hat{g}_k = (1/N) \cdot \sum_{r=0..N-1} g_{\{r,k\}}$$

脚本每个 micro-step 执行 `(loss/accum).backward()`，于是：

- 前 $K-1$ 步 (`no_sync` 上下文内)：PyTorch 的 `.grad` 是累加语义，本地缓冲里依次叠加 $(1/K) \cdot g_{\{r,1\}}$, $(1/K) \cdot g_{\{r,2\}}$, $\dots$ ，一次通信都不发；
- 第 K 步（恢复正常 backward）：触发 all-reduce，它同步的对象是 `.grad` 缓冲的当前值——里面已经装着前 $K-1$ 份，本次再叠加 $(1/K) \cdot g_{\{r,K\}}$ 后一起求和平均：

$$\begin{aligned} \text{all-reduce 后} &= (1/N) \cdot \sum_r \sum_{k=1..K} (1/K) \cdot g_{\{r,k\}} \\ &= (1/K) \cdot \sum_{k=1..K} \hat{g}_k = \nabla L_{\text{big}} \end{aligned}$$

正是目标大 batch 的梯度

为什么能省 $K-1$ 次通信：all-reduce 是线性算子，“每步通信、通信完再累加”与“先本地累加、最后一次通信”结果相同——通信的内容（各 rank 之和）不因延后而改变，所以延后合法；延后 $K-1$ 步，就是省掉 $K-1$ 次。反过来，第 K 步那一次不能省：optimizer.step 前必须拿到全体平均，这就是 `is_last` 分支存在的全部原因（忘了切回正常 backward = 梯度永远是本地值，训练静默走歪，不报错）。

实测收益：累积 2 步时通信量减半（脚本 02 默认 accum=2）。

5. 实测 (RTX 4090 $\times$ 2, torch 2.6.0+cu124, 脚本 02)

```

world_size=2: 每 rank batch=16  $\times$  accum=2  $\rightarrow$  有效 batch = 64
本 rank 吞吐: ~77k - 95k tokens/s (run-to-run 波动大, 取 ~90k; 平均 loss  $\approx$  3.6)
world_size=1: 有效 batch = 32
本 rank 吞吐: ~180k - 185k tokens/s (平均 loss  $\approx$  3.3)

```

（实测环境：RTX 4090 $\times$ 2, torch 2.6.0+cu124, NCCL；wall $\approx$ 0.2s 的 toy 规模，吞吐数字每次运行波动较大，仅供量级对照。）

• 怎么读这组数字：单卡 tokens/s 是“一张卡的产能”；双卡世界总产能 $\approx 90k \times 2 = 180k$ ，与单卡 $\sim 183k$ 基本持平（甚至略低）——因为这个 toy 模型太小（计算 0.2s 就完了，启动/通信占比高）。真实大模型上多卡才能体现价值；小模型测分布式加速比是新手常见误区。

（nanotron 的基准也是用真实规模模型测 MFU，而不是 toy。）

• 另一个正确观察：双卡 loss (≈ 3.6) 与单卡 (≈ 3.3) 不同——有效 batch 变大 + 数据分片不同，loss 不可直接比；要对比请固定有效 batch 与 sampler 种子做对照实验。

学完本章你能...

- 不看资料写出 DDP 五件套，并说出两个高频 bug（shuffle 双打乱、忘 set_epoch）
- 推导"all-reduce 平均 == 大 batch 梯度"，指出三个前提（等大分片/均值 loss/同起点）
- 画出桶化 all-reduce 与 backward 重叠的时序，解释"多卡近线性加速"从哪来
- 正确组合梯度累积与 no_sync，说清为什么省的是 K - 1 次且最后一次不能省
- 对"多卡吞吐数字"保持警惕：toy 模型的加速比不可信，看 MFU 要用真实规模

课后练习

```
<details>
<summary>Q1: DDP 构造时为什么要 broadcast 参数？不广播会怎样？</summary>
A: 保证所有 rank 从同一份权重出发。不广播的话各 rank 随机初始化各练各的——虽然每步 all-reduce 平均梯度理论上会逐渐趋同，但早期"观点分裂"浪费算力，且有 BN 统计量等状态不一致的实际问题。广播一次换来全程一致，成本可忽略。
</details>

<details>
<summary>Q2: 梯度累积 accum=4 时，loss 要不要除以 accum？为什么脚本里除的是 (loss/accum).backward()？</summary>
A: 要。PyTorch 的 backward 是把梯度【累加】到 .grad 上，累积 4 个 micro-batch 的目的是模拟"一个 4 倍大的 batch"——大 batch 的梯度是子 batch 梯度的平均，所以每个 micro-batch backward 前先除以 accum（或最后统一除）。不除的话等效学习率放大约 4 倍。
</details>

<details>
<summary>Q3: DDP + BatchNorm 有什么特殊行为？为什么 LayerNorm 没这个问题？</summary>
A: BN 的 running stats 是 buffer：DDP 默认每次 forward 前 broadcast rank0 的 buffer 同步它。但注意 BN 的 batch 统计仍是各 rank 自己的 batch 的（跨卡不同步统计，除非用 SyncBN）。LayerNorm 按样本内归一化、没有 running stats，天生无此问题——这也是现代 LLM 全用 LN/RMSNorm 的工程红利之一（呼应 Part 7 RMSNorm 一章）。
</details>
```

动手实践（依托脚本 02；动手 2 纯 CPU 可做）

动手 1：扫描 micro-batch × accum 组合——吞吐和 loss 到底哪个会变

任务：复制 `scripts/02_ddp_gpt.py` 为 `02_lab.py`，固定有效 batch = 64（world_size=2），跑三组组合并填表：

组合	batch_size	accum	有效 batch	每次 optimizer.step 的 all-reduce 轮数
A	32	1	64	1（每个 micro-step 都同步）
B（脚本默认）	16	2	64	1（前 1 步 no_sync）
C	8	4	64	1（前 3 步 no_sync）

步骤提示：

1. 每组只改两行：`batch_size=16` 与 `accum = 2`；
2. 打印行已经是变量拼接（`batch × accum × world`）——改完上面两行参数它会自动跟上，

无需再手改（顺带的小教训：日志要跟着参数走，硬编码的输出会"说谎"）；
3. `torchrun --standalone --nproc_per_node=2 02_lab.py` 每组跑一遍，抄"平均 loss"和
"本 rank 吞吐"两行。

验收标准：

- [] 三组打印的有效 batch 都是 64 ($bs \times accum \times world$ ，打印行用变量拼接、自动对上)
- [] 三组的平均 loss 相差 < 0.1 ——同有效 batch 下梯度数学等价（§ 2/§ 4），残差只来自浮点求和顺序与不同的数据分组。本机实测（RTX 4090 $\times$ 2, torch 2.6.0+cu124, NCCL）：
A/B/C = 3.687 / 3.607 / 3.617
- [] 吞吐落在 § 5 的 77k – 95k tokens/s 带内（本机实测 86.9k / 90.1k / 94.0k——accum 越大通信越少、吞吐略升，但 toy 规模下差距与 run-to-run 波动同量级）。能说出"这个实验看不出通信收益、要换什么规模才看得出"，也算过关。

动手 2：10 行代码验证 § 2——"平均的平均 == 总平均"，以及不等大时翻车

任务：验证等大分组的"平均的平均"等于总平均；再构造不等大分组看误差冒出来；最后用样本数加权救回来。单进程 CPU 即可（fp32）。

步骤提示（骨架）：

```
python
import torch, torch.nn as nn
torch.manual_seed(0)
lin = nn.Linear(4, 4)

def grad_of(x, y): # 返回这次 backward 后的权重梯度
    lin.zero_grad()
    nn.functional.cross_entropy(lin(x), y).backward()
    return lin.weight.grad.clone()

x, y = torch.randn(16, 4), torch.randint(0, 4, (16,))
g_full = grad_of(x, y) # 16 个样本一次算：总平均（基准）
```

Step 1: 等大两半 8+8 $\rightarrow (g1+g2)/2$ ，打印 $\max|\Delta|$ 与 `allclose(atol=1e-6)`

Step 2: 不等大两半 4+12 $\rightarrow$ 同样等权平均，打印 $\max|\Delta|$

Step 3: 不等大但按样本数加权 $(4 \cdot g1 + 12 \cdot g2)/16$ ，再打印 $\max|\Delta|$

验收标准：

- [] 等大 8+8： $(g1+g2)/2$ 与 `g_full allclose(atol=1e-6)` 为 True（本机实测 $\max|\Delta| \approx 1.5e-8$ ，CPU fp32）
- [] 不等大 4+12 等权平均： $\max|\Delta|$ 跳到 ~ 0.1 量级（本机实测 0.118，比等大情形大约 7 个数量级）——4 个样本占了 50% 的权重
- [] 不等大但按样本数加权： $\max|\Delta|$ 回到 $1e-8$ 量级、`allclose` 为 True——说清 § 2 表格里"等大"前提平时替你兜住了什么（DistributedSampler 为什么要切得等大）

课后作业

[Assignment 10](../..../assignments/assignment_10/) 题 3 (亲手证明 DistributedSampler 不重不漏)

下一步

DDP 解决"数据装不下", 但模型状态 (参数+梯度+优化器) 仍然是每卡一份。
7B 模型 $\times$ 16 字节/参数 = 112GB——下一章算清这本账, 并用 ZeRO/FSDP 把它切开。

[03 — 显存账本与 ZeRO/FSDP](03_memory_zero_fsdp.md)

03_memory_zero_fsdp

03 — 显存账本与 ZeRO / FSDP

> "7B 模型要用几张卡训?"能答对的人都会先算一笔账。本章先教会显存账本
> (脚本 03 的公式 + 逐字节模拟可复算), 再用 FSDP 实测分片效果 (脚本 04)。

学习目标

完成本章后, 你将能够:

- 背出 16Ψ 的五项构成 (参数/梯度/master/动量/方差), 现场心算 7B/70B 的 DDP 显存
- 推导 ZeRO 三阶段的每卡公式: $4\Psi + 12\Psi/N \rightarrow 2\Psi + 14\Psi/N \rightarrow 16\Psi/N$, 及各自的通信代价
- 写出 FSDP1 与 FSDP2 的最小用法, 解释 all-gather / reduce-scatter 在其中的角色
- 应用决策树为任意模型/卡数选并行策略 (DDP $\rightarrow$ ZeRO-1 $\rightarrow$ ZeRO-2 $\rightarrow$ FSDP / ZeRO-3)

前置知识

必须掌握:

- 02 章: DDP (每卡完整模型, 只同步梯度) ——本章的"对照面"

建议掌握:

- Part 8: AdamW、bf16 混合精度 (16Ψ 账本会用到, 本章也会当场重推)

1. 模型状态显存: 16 字节/参数

混合精度 (bf16/fp16 + fp32 master) AdamW 下, 每个参数要存:

项	字节
bf16 参数	2
bf16 梯度	2
fp32 master 权重	4

项	字节
fp32 动量 m	4
fp32 方差 v	4
合计	**16Ψ** (Ψ=参数量)

模型	DDP (每卡 16Ψ)	ZeRO-1 (N=8)	ZeRO-3 (N=8)
7B	112 GB	38.5 GB	14 GB/卡
70B	1120 GB	385 GB	140 GB/卡

• 这就是"为什么 7B 在 24G 卡上训不动"的完整答案：哪怕 bf16 存参数只要 14GB, AdamW 状态一加就到 112GB。ZeRO 的全部思想就一句话：****这 16Ψ 里大部分是优化器状态, 没必要每个卡都存完整份****。

ZeRO 三阶段 (Rajbhandari et al. 2019)：

阶段	切什么	每卡模型状态	通信代价
ZeRO-0 (DDP)	什么都不切	16Ψ	基准
ZeRO-1	优化器状态	4Ψ + 12Ψ/N	与 DDP 相同
ZeRO-2	+梯度	2Ψ + 14Ψ/N	与 DDP 相同
ZeRO-3 / FSDP	+参数	16Ψ/N	≈1.5× DDP

[脚本 03](../scripts/03_zero_memory.py) Part B 把每个 rank 持有的每一类张量逐字节加起来, 断言 ZeRO-1 与公式 $4\Psi + 12\Psi/N$ 一致 (ZeRO-2/3 的逐字节复算留作本章动手 2) ——面试被追问 "12Ψ/N 怎么来的"时, 这就是你的证据链。

• △ 激活值不在这本账里：激活 $\approx sbh \times (34 + 5 \cdot a \cdot s/h)$ 字节/层 (s=seq, b=batch, h=hidden, a=heads)，随并行方式无关、随 seq² 有平方项。省激活靠梯度检查点 (只存层输入、反传时重算, 显存从每层全存降到 2sbhL, 代价 $\approx$ 多一次 forward, 即 +33% 计算)。

2. FSDP：ZeRO-3 的 PyTorch 原生实现

DDP vs FSDP 一句话：DDP 每 rank 全量存储、只 all-reduce 梯度；FSDP 把参数/梯度/优化器状态全分片, forward 前 all-gather 收齐当前层的参数、算完即释放, backward 用 reduce-scatter 收梯度分片 —— 用 1.5× 通信换"单卡装得下"。

[脚本 04](../scripts/04_fsdp_gpt.py) (双卡实测)：

```
完整模型参数量: 11.8 MB (fp32)
FSDP 包装后本 rank 参数显存: 6.0 MB (全量 5.9 MB；≈一半参数分片 + FSDP 运行时缓冲)
训练后平均 loss: 2.981 (各 rank 初始化不同 → 广播同步正确)
```

```
python
```

FSDP1 API (torch 2.x 全可用；本课脚本用这个)

```
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
fsdp_model = FSDP(model, auto_wrap_policy=wrap_policy, # 每个 Block 一个分片单元
sharding_strategy=ShardingStrategy.FULL_SHARD, # = ZeRO-3
use_orig_params=True)
```

FSDP2 (torch 2.6+ 推荐, API 更干净; 概念完全一致)

```
from torch.distributed.fsdp import fully_shard
for layer in model.blocks:
    fully_shard(layer) # 先子模块
fully_shard(model) # 后根模块 (参数变 DTensor, 优化器不用改)
```

> 上面是工业写法 (按 Block 切分分片单元)。本课 [脚本 04](../scripts/04_fsdp_gpt.py)
> 为简洁起见用整体包裹 (不传 `auto_wrap_policy`, 分片单元 = 整个模型) —— 数学完全一致,
> 工业上用 `auto_wrap_policy` 按 Block 切分, all-gather 粒度更细、峰值显存更低 (见下方)。

- △ 版本提示: FSDP1 已被官方标记弃用, 新项目推荐直接用 FSDP2 的 `fully_shard`; 但网上大量代码 (和本课脚本) 是 FSDP1 —— 读得懂两者, 迁移只是换 API。
- 实践分工: 显存够用就 DDP (通信最少); 装不下再 ZeRO-1 (免费午餐); 还不够一路 ZeRO-3/FSDP。transformer_auto_wrap_policy 让每个 Block 成为独立分片单元, all-gather 粒度更细、峰值显存更低。

3. 决策树 (面试可直接背)

模型状态 16Ψ 装得下?

└— 是 → DDP (最优通信)

└— 否 → 先 ZeRO-1 (免费, 通信不变) → 再不够 ZeRO-2 → 还不够 FSDP/ZeRO-3

激活也爆? → 梯度检查点 + 减 micro-batch (配梯度累积保 effective batch)

单层都放不下? → 张量并行 TP (04 章)

学完本章你能...

- 背出 16Ψ 的五项构成, 现场算 7B/70B 的 DDP 显存
- 说出 ZeRO 三阶段各切什么、通信代价怎么变
- 写出 FSDP1 与 FSDP2 的最小用法, 解释 all-gather/reduce-scatter 在其中的角色
- 用"决策树"为任意模型/卡数选并行策略
- 扩展脚本 03 的账本函数复算 ZeRO-2/3, 并代入 7B/70B 填出三档对照表 (动手 1/2)

课后练习

<details>

<summary>Q1: ZeRO-1 为什么"免费" (通信量与 DDP 相同)? 切掉的 $12\Psi/N$ 里有什么? </summary>

A: fp32 master/动量/方差只在 optimizer.step() 时用, 且每个 rank 只 step 自己分片的参数 (参数全量在, 梯度全量在, 只对分片做更新再原地写回)。通信路径上没有任何新增集合操作。切掉的是 12 字节/参数的优化器状态 —— 它们是纯粹的"存储负担", 不参与前反向计算。

</details>

<details>

<summary>Q2: FSDP 的 all-gather 为什么能"算完就释放"? 什么配置下不释放? </summary>

A: 前向只需当前层的参数: FSDP 按 Block 为单元 all-gather → 算完该层即可释放, 下一个层再收下一份 (reshard_after_forward=True)。若设 False (=SHARD_GRAD_OP/ZeRO-2 语义), 参数收齐后整个 forward+backward 期间保留 —— 用显存换通信, 第一层之后的层不用重复收。

</details>

<details>

<summary>Q3: 7B 模型、8×24GB 卡。模型状态 112GB → ZeRO-3 每卡 14GB, 能训了吗? 还差什么? </summary>

A: 只能说"模型状态装得下了"。还要算: ① 激活 (seq/batch 大时轻松几十 GB → 梯度检查点); ② 通信是否成为瓶颈 (ZeRO-3 的 1.5×); ③ NCCL 通信缓冲与碎片余量 (实际可用 <24GB)。工业答案: 8 卡训 7B 常配 ZeRO-2/3 + 梯度检查点 + bf16 + flash-attention。

</details>

动手实践 (依托脚本 03, 纯 CPU 即可)

动手 1: 把账本代入 7B / 70B, 填出三档显存对照表

任务: 复制 `scripts/03_zero_memory.py` 为 `03_lab.py`, 把 Part A 的 `models` 列表换成 7B 与 70B, 并仿照 `s1` 那行的算式补打 ZeRO-2、ZeRO-3 两列 (公式就印在它下面四行)。

步骤提示:

1. $s2 = (2\psi + 14\psi / n) / 1e9$ 、 $s3 = (16 * \psi / n) / 1e9$, 照抄 `s1` 的

格式各加一格;

2. 跑 `python 03_lab.py`, 把 N=8 那列抄进你自己的表。

验收标准:

- [] N=8 时你的表与教程 §1 的表对得上, 并补齐它没列的 ZeRO-2 一档: 7B: DDP 112 / ZeRO-1 38.5 / ZeRO-2 26.25 / ZeRO-3 14 GB; 70B: 1120 / 385 / 262.5 / 140 GB (GB 按 $1e9$ 字节口径, 与脚本一致)
- [] 任意 $N > 1$ 都有 $\text{ZeRO-2} < \text{ZeRO-1}$: 两式相减 $(2\psi + 14\psi / N) - (4\psi + 12\psi / N) = (2/N - 2) \cdot \psi < 0$, 与 assignment 题 2 的"恒等式预警"互相印证
- [] 用你的表回答: 8 卡从 DDP 换 ZeRO-3, 每卡模型状态降到几分之一? ($1/8$, 与 N 无关的整数倍关系只在 ZeRO-3 成立——另外两档不是纯整除)

动手 2: 把逐字节模拟扩展到 ZeRO-2/3 (教程只替你断言了 ZeRO-1)

任务: 给 `zero_accounting_simulation` 加 `stage` 参数: `stage2` 把"bf16 梯度"那一行也改走 `shard_numel` (参数仍全量); `stage3` 把三类全改走 `shard_numel`。对 $N=2$ 与 $N=3$ 各断言一次。

步骤提示: 每类张量就是一行 `b += ...`——把"全量"的 `p.numel()` 换成"分片"的

`shard_numel(p.numel())` 就是升一阶。ZeRO 的"阶段"在代码里就是"多切一类张量",

这就是三行代码讲完的 ZeRO。

验收标准:

- [] `stage2`: $\text{sum}(\text{per_rank})/N$ 与公式 $2\psi + 14\psi / N$ 严格相等 (分片不重不漏 → 平均值精确对上; 对照公式别用 `==`, 用 `abs(a-b) < 1e-6`)
- [] 单个 rank 与公式的偏差不为 0 的情形: $N=3$ 时能看到百字节量级的 rank 间差 (本机实测 `stage2 @ N=3`: +233 / -103 / -131 字节) ——"公式是均摊值"的直观含义
- [] `stage3`: $\text{sum}(\text{per_rank})/N == 16\psi / N$, 且 $N=1$ 时 `stage 0/1/2/3` 四条全部退化为 16ψ (assignment 题 2 的"自检锚点"同款)
- [] 用脚本里的 TinyGPT ($\psi=6,298,624$) 复核一个数: `stage3 @ N=2` 每卡 ≈ 50.4 MB ($16\psi/2$; 对照 Part B 打印的 $16\psi = 100.8$ MB)

课后作业

[Assignment 10](../assignments/assignment_10/) 题 2 (显存账本计算器——公式驱动, 自动验算 ZeRO 各阶段)

下一步

到此为止每一层还是完整的。如果单层都放不下 (70B 的 FFN 一层就有几个 GB), 就要把计算本身切开: 张量并行与流水线并行 (进阶可选章)。

[04 — 张量并行、流水线并行与工业栈](04_tp_pp_and_beyond.md)

04_tp_pp_and_beyond

04 — 张量并行、流水线并行与工业栈 (进阶可选)

- > 最后一章是"进阶可选": 前两节的并行把模型状态切开了, 但如果单层权重/激活就装不下,
- > 要把一层之内的计算切开 (张量并行 TP), 或者让不同的卡负责不同的层 (流水线并行 PP)。
- > 这两种并行的代码复杂度显著更高——理解机制 + 亲手验证过一次最小实现, 就是本章的合格线;
- > 不要求在生产里自己写 TP/PP (那是 Megatron/DeepSpeed/nanotron 的工作)。
- >
- > 本章数字与公式全部在脚本 05/06 中实测可复现。

学习目标

完成本章后, 你将能够:

- 画出 Megatron MLP 的列/行并行切法与 f/g 共轭算子, 解释"每层每方向恰一次 all-reduce"
- 推导 $\text{bubble} = (p-1)/(m+p-1)$, 比较 GPipe 与 1F1B 的激活驻留 (m 份 vs p 份)
- 验证"按层切开不改变数学": 脚本 06 流水线 loss 与单进程一致到 $\sim 1e-6$
- 复述 3D 并行分工与"读配置"顺序: 机内 TP 优先 $\rightarrow$ PP 权衡气泡 $\rightarrow$ 剩余给 DP

前置知识

必须掌握:

- 03 章: ZeRO/FSDP 的通信原语 (all-gather / reduce-scatter)

建议掌握:

- 脚本 05/06: 本章的实证载体 (torchrun 双卡验证, 单进程也兼容)
- [Part 7 03 章](../Part7_minimind/tutorial/03_gqa_and_ffn.md): TP"按注意力头切分"引用这里的多头结构

1. 张量并行 (Megatron 式): 把一层切成两半

以 MLP $Y = \text{gelu}(X \cdot W1^T) \cdot W2^T$ 为例, Megatron 的切法:

W1（第一层）列并行：按输出维切 → 各rank算 $\text{gelu}(X \cdot W1_r^T)$ ，前向无通信
W2（第二层）行并行：按输入维切 → 各rank算 $H_r \cdot W2_r^T \rightarrow \text{all-reduce}$ 求和 = 完整 Y

- 为什么能"中间无通信"？因为 gelu 作用在分片内部：H 的列被切开，每列的计算只依赖 W1 的对应行块。两个共轭算子：f（forward 恒等 / backward all-reduce）与 g（forward all-reduce / backward 恒等）→ 每层 forward 恰 1 次、backward 恰 1 次 all-reduce。
- [脚本 05](../scripts/05_tensor_parallel.py) 实测 (2×4090)：

前向 $\max |Y_{tp} - Y_{dense}| = 5.96e-07 \rightarrow (< 1e-5 \text{ 验收线})$
梯度 $\max |dX_{tp} - dX_{dense}| = 5.82e-11 \rightarrow$

- Attention 同构：QKV 投影按头切（天然列并行，呼应 Part 7 GQA 的多头结构），输出投影行并行。
- ⚠ TP 的通信在每层内部，频率极高 → 只适合 NVLink 互联的同一台机器内（机内 8 卡），跨机用 TP 会把带宽吃光。进阶：all-reduce 还能拆成 reduce-scatter + all-gather，顺带把 LayerNorm/Dropout 的激活也分片（sequence parallelism, arXiv 2205.05198）。

2. 流水线并行（GPipe / 1F1B）：按层接力

4 层模型、2 个 stage：

stage0 (rank0)：embedding + blocks[0:2] stage1 (rank1)：blocks[2:] + head

数据切成 $m=4$ 个 micro-batch 填流水线：

stage0: F1 F2 F3 F4 ————— B4 B3 B2 B1

stage1: F1 F2 F3 F4 ————— B4 B3 B2 B1 (F=forward, B=backward)

↑ 气泡 ↑

- [脚本 06](../scripts/06_pipeline_parallel.py) 用两个自定义 autograd.Function（Send/RecvActivation, forward 传激活、backward 传梯度）实现了最小 GPipe，实测流水线 loss 与单进程整模型完全一致 ($4.380254 == 4.380254$) —— "按层切开不改变数学"的最硬证据。
- bubble 公式：气泡占比 = $(p-1)/(m+p-1)$ 。 $p=2, m=4 \rightarrow 20\%$ ；m 增大气泡被摊薄，但 m 个 micro-batch 的激活也要驻留（GPipe）；1F1B 调度交错执行 forward/backward，把激活驻留从 m 个降到 p 个——大模型流水线的标配。
- ⚠ 工程实测坑（本课开发机踩到）：4090+4090D 混合机型上 NCCL 的 send/recv 点对点会互相卡死（集合通信正常）。脚本 06 的解法：点对点单独建 gloo 组、CPU 中转。
教训：分布式问题不总是逻辑 bug，通信后端与硬件拓扑的组合也要怀疑。

3. 拼起来：3D 并行与工业栈

总卡数 = $TP \times PP \times DP$ （自检：乘积必须等于总卡数，如 $TP=8 \times PP=2 \times DP=2 = 32$ 卡）
LLaMA 2 70B 官方报告用 2000+ 卡、 $MFU \approx 46\%$ ——具体拆法未完全公开，按下面的习惯推（读配置的习惯：TP 尽量小且机内 → PP 其次 → 剩下全部给 DP；再叠 ZeRO-1/激活重计算）

工业参考栈（按"想继续深入"排序）：

源	适合谁
nanotron](https://github.com/huggingface/nanotron)	想读"能跑通的 3D 并行极简实现"（PP 有 AFAB/1F1B, ZeRO-1, MoE 专家并行）
Ultra-Scale Playbook](https://huggingface.co/spaces/nanotron/ultrascale-playbook)	想系统学"集群训 LLM"（530 卡 Llama-1B 全过程讲解，本课的下一站）
nanotron-LM / DeepSpeed	生产级 TP/PP/ZeRO 的原始出处（读文档理解语义即可，不必上手）
PyTorch	PyTorch 官方的 LLM 预训练脚手架（FSDP2 + 并行组合的现成配置）

学完本章你能...

- 画出 MLP 的列/行并行切法，解释 f/g 算子与"每层恰 2 次 all-reduce"
- 画出 GPipe 时间线，算 $\text{bubble} = (p - 1) / (m + p - 1)$ ，说出 1F1B 省了什么
- 复述 TP 适合机内（NVLink）、PP 消息少但气泡大、DP 最便宜的分工逻辑
- 说出"读配置"的顺序：先 TP 后 PP 剩 DP，再叠 ZeRO 与激活重计算

课后练习

<details>
<summary>Q1: 为什么 TP 的 all-reduce 不能像 DDP 那样与计算重叠，导致它对带宽最敏感？</summary>
A: DDP 的梯度 all-reduce 在 backward 尾声、可按桶异步化，与"其余层反传"重叠；TP 的 all-reduce 在【每一层的正中间】，前后都是依赖它的计算，遮不住。所以 TP 只放机内（NVLink ~900GB/s 级），跨机（~25-100GB/s）会被通信吃掉大半 MFU。
</details>

<details>
<summary>Q2: $p=8$ 个 stage，想让 $\text{bubble} < 10\%$ ，micro-batch m 至少多大？代价是什么？</summary>
A: $(p-1)/(m+p-1) < 0.1 \rightarrow m+p-1 > 70 \rightarrow m \geq 63$ 。代价：GPipe 下 63 个 micro-batch 的激活都要驻留 $\rightarrow$ 激活显存爆炸，所以要换 1F1B（激活驻留降到 p 个）+ 梯度检查点。
</details>

<details>
<summary>Q3: 4×8 卡（机内 4 台、每台 8 卡）训一个 70B，你会怎么分配 TP/PP/DP？</summary>
A: $4 \text{ 台} \times 8 \text{ 卡} = 32 \text{ 卡}$ 。TP=8（占满机内 NVLink） $\rightarrow$ 剩 $32/8 = 4$ 个"流水线单元" $\rightarrow$ PP=2 $\times$ DP=2（或 PP=4 $\times$ DP=1），按激活显存与气泡权衡（配 1F1B）；DP 部分叠 ZeRO-1。
这不是唯一解——面试重点是给出"机内 TP 优先、气泡与显存权衡、剩余给 DP"的推理链。
</details>

课后作业

[Assignment 10](../..../assignments/assignment_10/) 题 4（TP 分块数学）+ 题 5（bubble 计算器）

全课程毕业

Part 1-5 会训练 Part 6-8 会造模型、会后训练
Part 9 懂硬件 Part 10 懂集群
下一站：nanotron / Ultra-Scale Playbook / llm.c —— 带着这套地基去读它们

[\[← 上一章：显存账本与 ZeRO/FSDP\]\(03_memory_zero_fsdp.md\)](#) | [\[Part 10 README\]\(README.md\)](#)